



SESSION 3

Loops and Selection Statements

Source:

- Kenneth A. Lambert, Fundamentals of Python: First Programs, 3rd Ed., Cengage, 2024. ISBN: 9780357881019

Objectives

By the end of this class, you should be able to:

- Write a loop to repeat a sequence of actions a fixed number of times
- Format text for output
- Use selection statements with Boolean expressions to make choices in a program
- Write loops that continue when a Boolean expression is true and halt when it becomes false

Definitive Iteration: The for loop

- Repetition statements (or **loops**) repeat an action
- Each repetition of an action is known as a **pass** or **iteration**
- Two types of loops
 - Those that repeat an action a predefined number of times (**definite iteration**)
 - Those that perform an action until the program determines it needs to stop (**indefinite iteration**)

Executing a Statement a Given Number of Times (1 of 2)

- Python's `for` loop is the control statement that most easily supports definite iteration

```
>>> for eachPass in range(4):  
    print("It's alive!", end = " ")  
It's alive! It's alive! It's alive! It's alive!
```

- The form of this type of loop is:

```
for <variable> in range(<an integer expression>):  
    <statement-1>  
    ...  
    <statement-n>
```

- First line of code is sometimes called the **loop header**
- Rest of code is the **loop body** (must be indented and aligned in the same column)

Executing a Statement a Given Number of Times (2 of 2)

- Example: Loop to compute an exponentiation for a non-negative exponent

```
>>> number = 2
>>> exponent = 3
>>> product = 1
>>> for eachPass in range(exponent):
        product = product * number
        print(product, end = " ")
2 4 8
>>>> product
8
```

- If the exponent were 0, the loop body would not execute and the value of `product` would remain 1

Count-Controlled Loops (1 of 2)

- **Count-controlled loops:** loops that count through a range of numbers

```
>>> product = 1
>>> for count in range (4):
        product = product * (count + 1)
>>> product
24
```

- To specify an explicit lower bound:

```
>>> product = 1
>>> for count in range (1, 5):
        product = product * count
>>> product
24
```

Count-Controlled Loops (2 of 2)

- Example: bound-delimited **summation**

```
>>> lower = int(input("Enter the lower bound: "))
Enter the lower bound: 1
>>> upper = int(input("Enter the upper bound: "))
Enter the upper bound: 10
>>> theSum = 0
>>> for number in range(lower, upper + 1):
        theSum = theSum + number
>>> theSum
55
```

Augmented Assignment

- Combine the assignment symbol with arithmetic and concatenation operators to provide **augmented assignment operations**:

```
a = 17
```

```
s = "hi"
```

```
a += 3          # Equivalent to a = a + 3
```

```
a -= 3          # Equivalent to a = a - 3
```

```
a *= 3          # Equivalent to a = a * 3
```

```
a /= 3          # Equivalent to a = a / 3
```

```
a %= 3          # Equivalent to a = a % 3
```

```
s += " there"   # Equivalent to s = s + " there"
```

- Format:** *<variable> <operator>= <expression>*
- Equivalent to:** *<variable> = <variable> <operator> <expression>*

Loop Errors: Off-by-One Error

- Example:

```
# Count from 1 through 4, we think
>>> for count in range(1,4):
        print(count)
```

```
1
2
3
```

- Loop actually counts from 1 through 3
- Not a syntax error, but rather a logic error

Traversing the Contents of a Data Sequence

- `range` returns a **list**

```
>>> list(range(4))
```

```
[0, 1, 2, 3]
```

```
>>> list(range(1, 5))
```

```
[1, 2, 3, 4]
```

- Strings are also sequences of characters
- Values in a sequence can be visited with a `for` loop:

```
for <variable> in <sequence>:
```

```
    <do something with variable>
```

- Example:

```
>>> for character in "Hi there!":
```

```
    print(character, end = " ")
```

```
H i   t h e r e !
```

Specifying the Steps in the Range

- `range` expects a third argument that allows you specify a **step value**

```
>>> list(range(1, 6, 1)) # Same as using two arguments
```

```
[1, 2, 3, 4, 5]
```

```
>>> list(range(1, 6, 2)) # Use every other number
```

```
[1, 3, 5]
```

```
>>> list(range(1, 6, 3)) # Use every third number
```

```
[1, 4]
```

- **Example:**

```
>>> theSum = 0
```

```
>>> for count in range(2, 11, 2):
```

```
    theSum += count
```

```
>>> theSum
```

```
30
```

Loops that Count Down

- Example:

```
>>> for count in range(10, 0, -1):  
        print(count, end = " ")  
10 9 8 7 6 5 4 3 2 1  
>>> list(range(10, 0, -1))  
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Activity 1: Knowledge Check

1. What is the augmented assignment operator for string concatenation?
2. What is output by the following loop?

```
>>> for count in range(0, 2):  
    print(count)
```
3. What is the third argument in the `range` function?

Activity 1: Knowledge Check Answers

1. What is the augmented assignment operator for string concatenation?

Answer: +=

2. What is output by the following loop?

```
>>> for count in range(0, 2):  
        print(count)
```

Answer:

```
0  
1
```

3. What is the third argument in the `range` function?

Answer: The step value, which is the interval between the numbers used in the range

Formatting Text for Output (1 of 3)

- Many data-processing applications require output that has a **tabular format**
- **Field width:** total number of data characters and additional spaces for a datum in a formatted string
- The `print` function automatically begins printing an output datum in the first available column

```
>>> for exponent in range(7, 11):  
        print(exponent, 10 ** exponent)  
7 10000000  
8 100000000  
9 1000000000  
10 10000000000
```

Formatting Text for Output (2 of 3)

`<format string> % <datum>`

- This version contains a **format string**, the **format operator** `%`, and a single data value to be formatted
 - To format integers, the letter `d` is used instead of `s`

- To format a sequence of data values:

`<format string> % (<datum-1>, ..., <datum-n>)`

```
>>> for exponent in range(7, 11):
```

```
    print("%-3d%12d" % (exponent, 10 ** exponent))
```

```
7      100000000
```

```
8     1000000000
```

```
9    10000000000
```

```
10   100000000000
```

Formatting Text for Output (3 of 3)

- To format data value of type `float`:

`%<field width>.<precision>f`

where `.<precision>` is optional

- Examples:

```
>>> salary = 100.00
```

```
>>> print("Your salary is $" + str(salary))
```

```
Your salary is $100.0
```

```
>>> print("Your salary is $%0.2f" % salary)
```

```
Your salary is $100.00
```

Selection: `if` and `if-else` Statements

- **Selection statements** allow a computer to make choices
 - Based on a **condition**

The Boolean Type, Comparisons, and Boolean Expressions

- **Boolean data type** consists of two values: true and false (Python Boolean literals are `True/False`)
- Example: `4 != 4` evaluates to `False`

Comparison Operator	Meaning
<code>==</code>	Equals
<code>!=</code>	Not equals
<code><</code>	Less than
<code>></code>	Greater than
<code><=</code>	Less than or equal
<code>>=</code>	Greater than or equal

if-else Statements (1 of 2)

- Also called a **two-way selection statement**

- Often used to check inputs for errors:

```
import math
area = float(input("Enter the area: "))
if area > 0:
    radius = math.sqrt(area / math.pi)
    print("The radius is", radius)
else:
    print("Error: the area must be a positive number")
```

- Syntax:

```
if <condition>:
    <sequence of statements-1>
else:
    <sequence of statements-2>
```

if-else Statements (2 of 2)

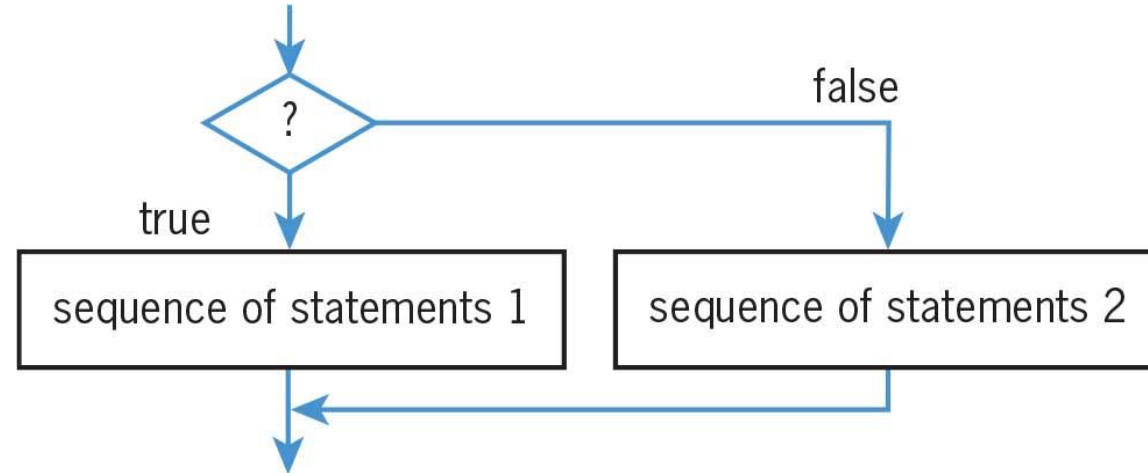


Figure 3-2 The semantics of the `if-else` statement

One-Way Selection Statements

- Simplest form of selection is the **if statement**

```
if <condition>:  
    <sequence of statements>
```

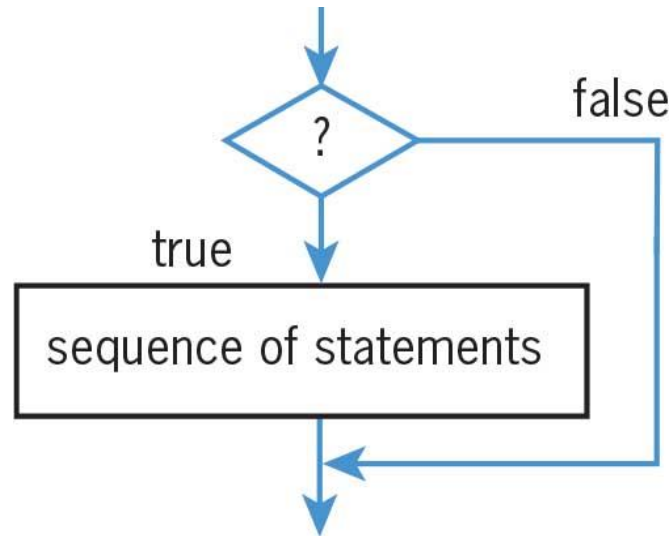


Figure 3-3 The semantics of the `if` statement

Multiway if Statements (1 of 4)

- A program may be faced with testing conditions that entail more than two alternative courses of action
- Can be described in code by a **multiway selection statement**

Letter Grade	Range of Numeric Grades
A	All grades above 89
B	All grades above 79 and below 90
C	All grades above 69 and below 80
F	All grades below 70

Multiway if Statements (2 of 4)

- Example:

```
number = int(input("Enter the numeric grade: "))
if number > 89:
    letter = 'A'
elif number > 79:
    letter = 'B'
elif number > 69:
    letter = 'C'
else:
    letter = 'F'
print("The letter grade is", letter)
```

Multiway if Statements (3 of 4)

- Syntax:

```
if <condition-1>:  
    <sequence of statements-1>  
elif <condition-n>:  
    <sequence of statements-n>  
else:  
    <default sequence of statements>
```

Multiway `if` Statements (4 of 4)

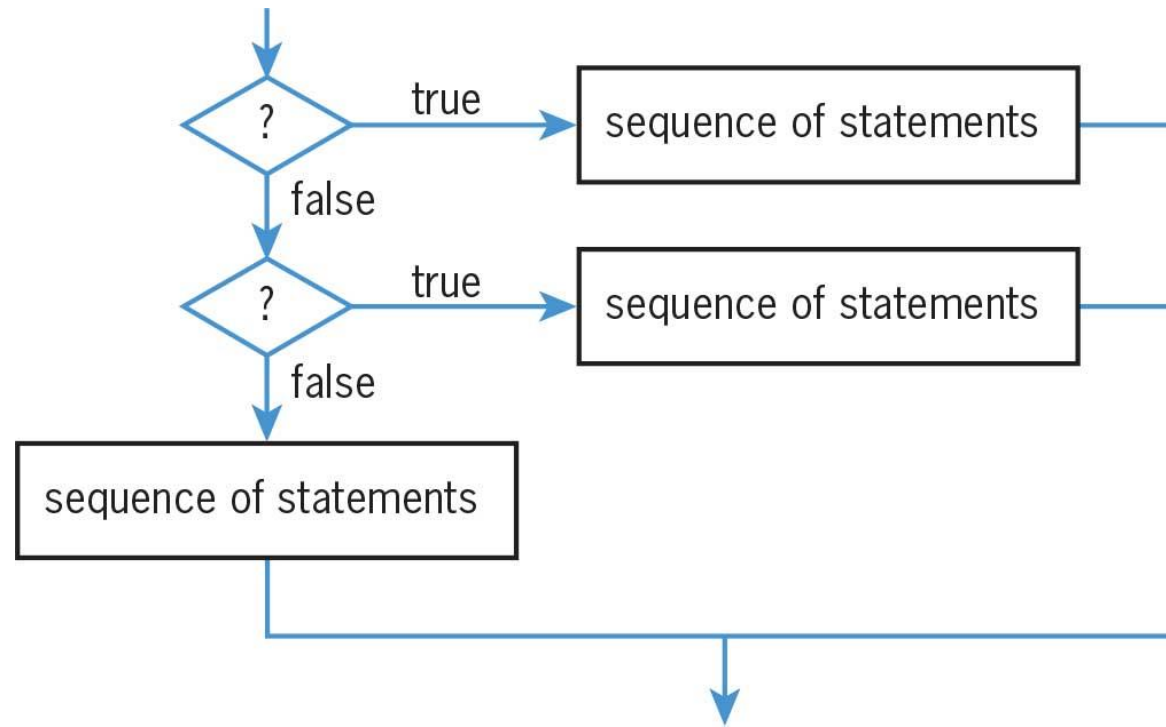


Figure 3-4 The semantics of the multiway `if` statement

Logical Operators and Compound Boolean Expressions (1 of 4)

- Often a course of action must be taken if either of two conditions is true. Below are two approaches:

```
number = int(input("Enter the numeric grade: "))
if number > 100:
    print("Error: grade must be between 100 and 0")
elif number < 0:
    print("Error: grade must be between 100 and 0")
else:
    # The code to compute and print the result goes here
```

- Simplified Code:

```
number = int(input("Enter the numeric grade: "))
if number > 100 or number < 0:
    print("Error: grade must be between 100 and 0")
else:
    # The code to compute and print the result goes here
```

Logical Operators and Compound Boolean Expressions (2 of 4)

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

A	not A
True	False
False	True

Figure 3-5 The truth tables for and, or, and not

Logical Operators and Compound Boolean Expressions (3 of 4)

- Next example verifies some of the claims made in the previous truth tables:

```
>>> A = True
>>> B = False
>>> A and B
False
>>> A or B
True
>>> not A
False
```

- The logical operators are evaluated after comparisons but before the assignment operator
 - `not` has higher precedence than `and`, which has higher precedence than `or`

Logical Operators and Compound Boolean Expressions (4 of 4)

Type of Operator	Operator Symbol
Exponentiation	**
Arithmetic negation	-
Multiplication, division, remainder	*, / , %
Addition, subtraction	+, -
Comparison	==, !=, <, >, <=, >=
Logical negation	not
Logical conjunction	and
Logical disjunction	or
Assignment	=

Short-Circuit Evaluation

- In `(A and B)`, if A is false, then so is the expression, and there is no need to evaluate B
- In `(A or B)`, if A is true, then so is the expression, and there is no need to evaluate B
- **Short-circuit evaluation:** evaluation that stops as soon as possible

- Example:

```
count = int(input("Enter the count: "))
theSum = int(input("Enter the sum: "))
if count > 0 and theSum // count > 10:
    print("average > 10")
else:
    print("count = 0 or average <= 10")
```

Testing Selection Statements

- Tips:
 - Make sure that all of the possible branches or alternatives in a selection statement are exercised
 - After testing all of the actions, examine all of the conditions
 - Test conditions that contain compound Boolean expressions using data that produce all of the possible combinations of values of the operands

Activity 2 Discussion

1. Are you familiar with the `switch` (or `case`) selection statements available in some programming languages? If so, do you think `switch` statements are more convenient than multiway `if` statements, or the other way around?
2. How would you go about developing a set of test cases for the following code snippet that satisfies all the testing tips from the previous slide?

```
count = int(input("Enter the count: "))
theSum = int(input("Enter the sum: "))
if count > 0 and theSum // count > 10:
    print("average > 10")
else:
    print("count = 0 or average <= 10")
```

Conditional Iteration: The `while` Loop

- The **`while` loop** can be used to describe **conditional iteration**
 - Example: A program's input loop that accepts values until the user enters a **sentinel** that terminates the input

The Structure and Behavior of a `while` Loop (1 of 3)

- Conditional iteration requires that the condition be tested within the loop to determine if it should continue
 - Called **continuation condition**
- Syntax:

```
while <condition>:  
    <sequence of statements>
```
- Improper use may lead to an **infinite loop**
- The `while` loop is also called an **entry-control loop**
 - Condition is tested at the top of the loop
 - Statements within the loop can execute zero or more times

The Structure and Behavior of a `while` Loop (2 of 3)

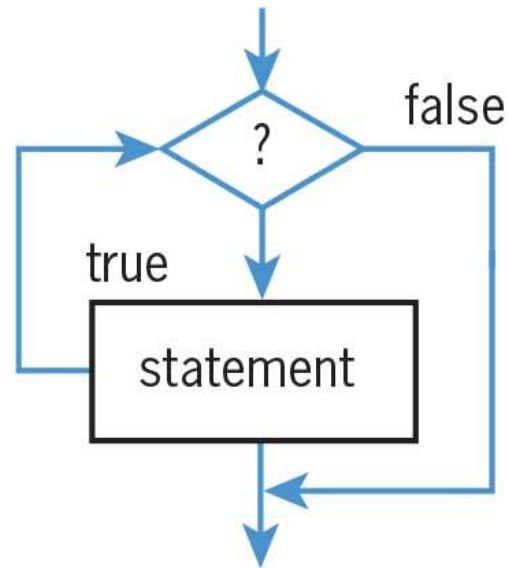


Figure 3-6 The semantics of a `while` loop

The Structure and Behavior of a while Loop (3 of 3)

- Example:

```
theSum = 0.0
data = input("Enter a number or just enter to quit: ")
while data != "":
    number = float(data)
    theSum += number
    data = input("Enter a number or just enter to quit: ")
print("The sum is", theSum)
Enter a number or just enter to quit: 3
Enter a number or just enter to quit: 4
Enter a number or just enter to quit: 5
Enter a number or just enter to quit:
The sum is 12.0
```

- data is the **loop control variable**

- First input statement initializes a variable to a value that the loop condition can test

Count Control with a while Loop

- You can use a while loop for a count-controlled loop

```
# Summation with a for loop
theSum = 0
for count in range(1, 100001):
    theSum += count
print(theSum)
```

```
# Summation with a while loop
theSum = 0
count = 1
while count <= 100000:
    theSum += count
    count += 1
print(theSum)
```

The while True Loop and the break Statement (1 of 3)

- The `while` loop can be complicated to write correctly; It is possible to simplify its structure and improve its readability

```
theSum = 0.0
while True:
    data = input("Enter a number or just enter to quit: ")
    if data == " ":
        break
    number = float(data)
    theSum += number
print("The sum is", theSum)
```

- Within this body, the input datum is received
- It is then tested for the loop's **termination condition** in a one-way selection statement
 - The `break` statement will cause an exit from the loop

The while True Loop and the break Statement (2 of 3)

- This example modifies the input section of the grade-conversion program to continue taking input numbers from the user until an acceptable value is entered:

```
while True:
    number = int(input("Enter the numeric grade: "))
    if number >= 0 and number <= 100:
        break
    else:
        print("Error: grade must be between 100 and 0")
print(number) # Just echo the valid input
```

The while True Loop and the break Statement (3 of 3)

- Alternative: Use a Boolean variable to control loop

```
done = False
```

```
while not done:
```

```
    number = int(input("Enter the numeric grade: "))
```

```
    if number >= 0 and number <= 100:
```

```
        done = True
```

```
    else:
```

```
        print("Error: grade must be between 100 and 0")
```

```
print(number) # Just echo the valid input
```

Random Numbers (1 of 3)

- Programming languages include resources for generating **random numbers**
- `random` module supports several ways to do this
 - `randint` returns a random number from among the numbers between two arguments, inclusive

```
>>> import random
>>> for roll in range(10):
        print(random.randint(1, 6), end = " ")
2 4 6 4 3 2 3 6 2 2
```

Random Numbers (2 of 3)

- Example: A simple guessing game

```
import random
smaller = int(input("Enter the smaller number: "))
larger = int(input("Enter the larger number: "))
myNumber = random.randint(smaller, larger)
count = 0
while True:
    count += 1
    userNumber = int(input("Enter your guess: "))
    if userNumber < myNumber:
        print("Too small!")
    elif userNumber > myNumber:
        print("Too large!")
    else:
        print("Congratulations! You've got it in", count, "tries!")
        break
```

Random Numbers (3 of 3)

- Output when this program is run:

```
Enter the smaller number: 1
Enter the larger number: 100
Enter your guess: 50
Too small!
Enter your guess: 75
Too large!
Enter your guess: 63
Too small!
Enter your guess: 69
Too large!
Enter your guess: 66
Too large
Enter your guess: 65
You've got it in 6 tries!
```

Loop Logic, Errors, and Testing

- Errors to rule out during testing of a `while` loop:
 - Incorrectly initialized loop control variable
 - Failure to update this variable correctly within the loop
 - Failure to test it correctly in the continuation condition
- To halt a loop that appears to hang during testing, type `Control+c` in the terminal window or IDLE shell
- If a loop must run at least once, use a `while True` loop with delayed examination of the termination condition
 - Ensure a `break` statement will be reached eventually

Summary

- **What we learned**

- How to write loops that repeat a sequence of actions a fixed number of times.
- How to format text for output, so program results are displayed in a clear and readable way.
- How to use selection statements with Boolean expressions to make decisions in a program.
- How to write conditional loops that continue running while a Boolean expression is true and stop once it becomes false.

- **Next class:** Working with strings and numbers